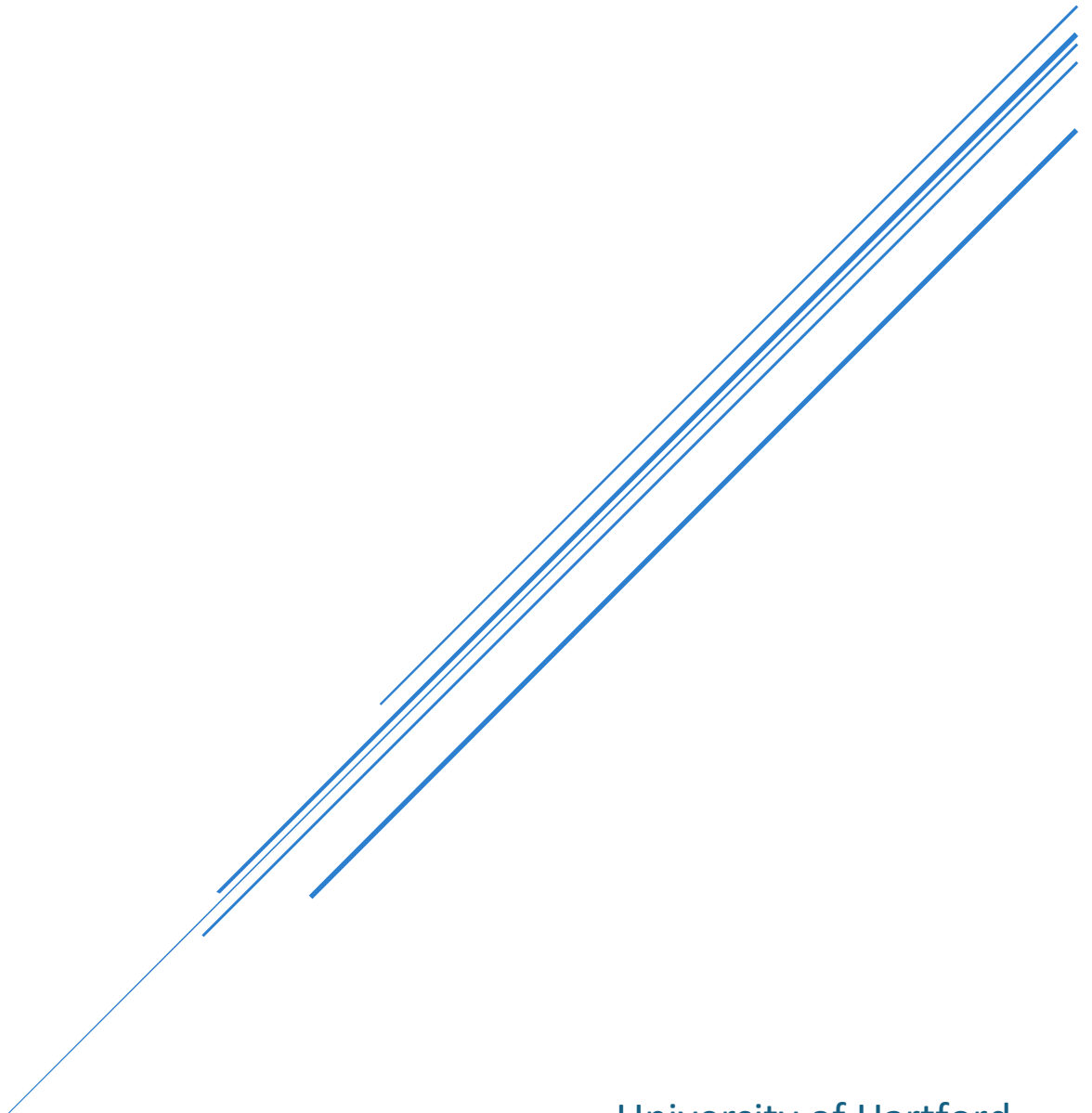


RESEARCH AND DEVELOPMENT OF AN OPEN-SOURCE ACOUSTIC CAMERA

Jesse Kaye & Nathan Grover



University of Hartford
ME 461 – Acoustic Capstone Design

LEGAL NOTICE

This report was prepared as part of a capstone design project at the University of Hartford. Neither the University nor the Engineering Applications Center, nor any person acting on behalf of the University:

A. Makes any warranty or representation, express or implied, with respect to the accuracy, completeness, or usefulness of information contained in this report, or that the use of any information, apparatus, method, or process disclosed in this report may not infringe on privately owned rights; or

B. Assumes any liability with respect to the use of, or for damages resulting from the use of any information, apparatus, method, or processes disclosed in this report.

As used in the above, "person acting on behalf of the University" includes any employee or contractor of the University to the extent that such employee or contractor prepares, handles or distributes, or provides access to any information pursuant to his employment or contract with the University.

Abstract

The development of an open-source acoustic camera is described and assembled using commercially available hardware along with freely accessible software, providing the potential for a more financially feasible kit assembly project. All developed code, components, and hardware are intended to be documented on a GitHub page along with full build instructions. The benefits and drawbacks of several digital beamforming methods, PCB design for audio purposes, and coding performant software on low performing machines are discussed. Preliminary results for a sample prototype will be presented. The performance, accuracy, and limitations of the design are assessed.

Table of Contents

Abstract	1
Table of Contents	2
Purpose	6
Background	6
History of Project	6
Acoustic Camera Market	6
Open-Source Software Ideology	7
Procedure with Theory	8
Hardware Design	8
Microphone Array Design	9
Mechanical Component Design	11
Software Design	18
Audio	18
Video Processing	21
GUI Overview and Operational Instructions	22
Software and Prerequisite Library Installation.....	25
Dependencies.....	25
OpenCV Installation	26
Compatibility	27
Results	29
Anechoic Chamber	29
Reverberation Room	30
Lecture Hall	31
Cost	33
Discussion and Conclusions	34
Performance.....	34
Precision and Accuracy	34
Design Oversights.....	35

Future Development Paths	36
References.....	38
Appendix	39

List of Figures

Figure 1: Hardware Overview Block Diagram	8
Figure 2: Quad-Channel, PDM Mic Array Schematic	10
Figure 3: Microphone Array PCB Design, Rear.....	11
Figure 4: Microphone Array PCB Design, Front	11
Figure 5: Mounting Plate for Camera and Arrays, Rear, with channels numbered	12
Figure 6: Assembled Mounting Plate	14
Figure 7: Camera Mounting Bracket	14
Figure 8: MCH Mounting Bracket.....	15
Figure 9: Screen Backplate	15
Figure 10: Mounting Plate with Screws for Pi.....	15
Figure 11: Mounting Plate with Screen Attached	16
Figure 12: Screen Bezel	16
Figure 13: Screen Bezel with Bolts	17
Figure 14: Screen Bezel with Screen Backplate installed	17
Figure 15: Screen Bezel assembled, alternate view.....	18
Figure 16: Audio Processing Overview.....	19
Figure 17: URA of Microphones Demonstration.....	20
Figure 18: Video Processing Overview.....	22
Figure 19: Acoustic Camera GUI.....	23
Figure 20: Acoustic Camera GUI: Info Menu, cropped and enlarged for legibility	23
Figure 21: Acoustic Camera GUI: Controls Menu	25
Figure 22: Acoustic Camera GUI: Options Menu	25
Figure 23: Anechoic Chamber Test, TL.....	30
Figure 24: Anechoic Chamber Test, TR	30
Figure 25: Anechoic Chamber Test, BL.....	30
Figure 26: Anechoic Chamber Test, BR	30
Figure 27: Reverberation Room Test, TL	31
Figure 28: Reverberation Room Test, TR.....	31
Figure 29: Reverberation Room Test, BL	31
Figure 30: Reverberation Room Test, BR.....	31
Figure 31: Lecture Hall, Longer Distance, Frame 1	32
Figure 32:Lecture Hall, Longer Distance, Frame 2	32
Figure 33:Lecture Hall, Longer Distance, Frame 3	32
Figure 34:Lecture Hall, Longer Distance, Frame 4	32
Figure 35:Lecture Hall, Longer Distance, Frame 5	32
Figure 36:Lecture Hall, Longer Distance, Frame 6	32

List of Tables

Table 1: Sample of Commercial Acoustic Camera Options	7
Table 2: Sliders and their Functions	24
Table 3: Options Menu Checkboxes and their Functions.....	24
Table 4: Options Menu Buttons and their Functions	24
Table 5: Required Dependencies	26
Table 6: OpenCV Installation Commands.....	27
Table 7: Raspberry Pi 5 Specifications.....	28
Table 8: Development Laptop Specifications	28
Table 9: Virtual Machine for Development Specifications.....	28
Table 10: Component Costs at Time of Purchase	33
Table 11: Bill of Materials for Each Array Board	39

Purpose

The primary purpose of this project was to develop an open-source acoustic camera, addressing the significant cost barrier associated with commercially available alternatives. This effort aimed to facilitate wider accessibility to acoustic imaging technology by leveraging freely available software libraries and accessible, affordable hardware components. Comprehensive documentation of the design, assembly, and software implementation was provided to enable straightforward replication and adaptation. This approach was intended to empower both acousticians and hobbyists with a cost-effective, DIY tool for acoustic analysis and investigation.

Background

History of Project

The project's initial ideation began around July 2024. Both phase difference of arrival (PDoA) and time difference of arrival (TDoA) beamforming methods have been under development throughout the project to assess the viability of each. Research focused on these methods, evaluating their performance alongside the perceived ease of implementation; a critical factor given the limited programming experience available for the project. Initially, PDoA was the primary method considered, due to its relative simplicity. Subsequently, in August, planning for the hardware commenced, alongside the learning of necessary libraries. By October, the design of the microphone arrays was finalized, boards and parts were ordered, and the arrays were assembled. A transition occurred in January 2025, moving from an OpenCV GUI to ImGui for development. February 2025 was the start of the development for the TDoA method of beamforming. Ongoing work continued with the refinement of controls throughout February, and finally, in March, report writing began, accompanied by minor adjustments to the video as well as the final switch back to the PDoA method of beamforming.

Acoustic Camera Market

Currently commercially available acoustic cameras are prohibitively expensive and limited in variety. Notable brands are HBK, FLIR and Fluke. Cost for various models at the time of writing is in tens of thousands of dollars, making such a tool inaccessible to many companies and individuals.

Table 1 lists some commercial options with prices at the time of writing (Spring 2025).

Table 1: Sample of Commercial Acoustic Camera Options

Brand and Model	List Price [USD]
MEGGER MPAX128 ATEX [1]	\$36,190.076
Fluke II915 [2]	\$24,998.99
NTI Sorama iV64 [3]	\$10,480.00

Open-Source Software Ideology

All resources for this project are to be made open-source. This includes the hardware designs, all code, and instructions for building or contributing to design. A GitHub page will accompany the acoustic camera using the MIT license, See Appendix.

Procedure with Theory

Hardware Design

The acoustic camera's hardware was chosen based on cost and availability. An overview of the system is shown below in Figure 1, featuring a Raspberry Pi 5, a MiniDSP MCH Streamer audio interface, four quad-channel microphone arrays, and a seven-inch, 1024 x 600 touch screen.

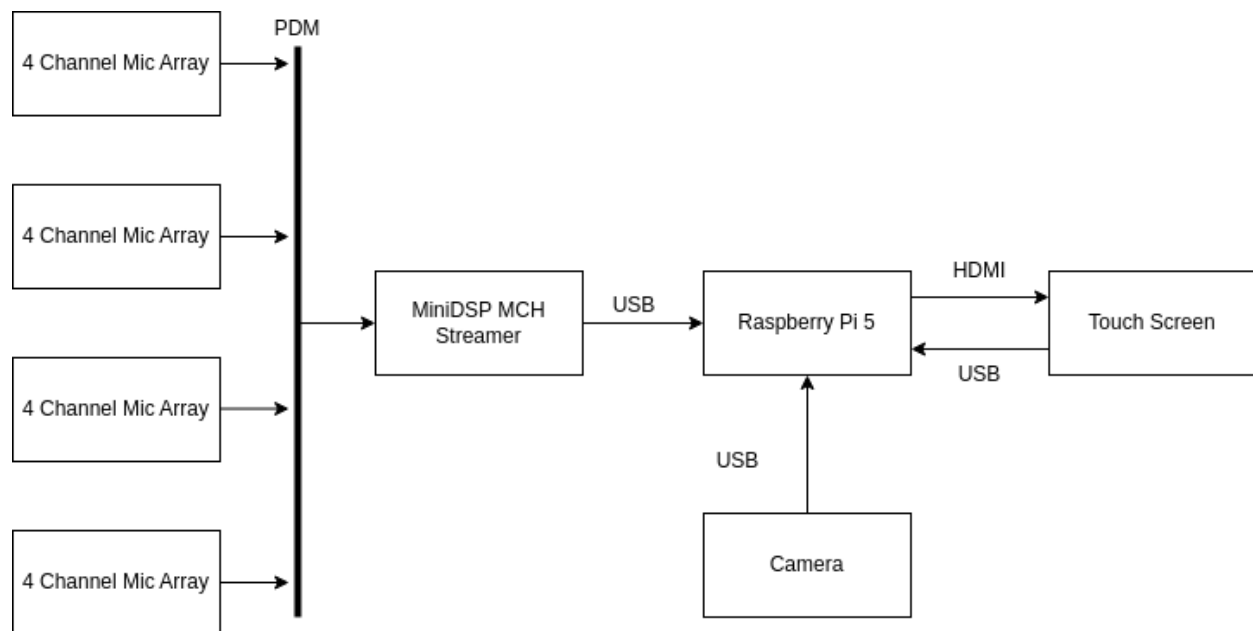


Figure 1: Hardware Overview Block Diagram

The Raspberry Pi 5 was selected as the single board computer, which offered compatibility, software and hardware support, and a cost-effective design. It boasts a quad-core ARM processor, eight gigabytes of RAM, and an SD card slot for storage. Raspberry Pi OS, Bookworm, (a Linux distribution intended for use with Raspberry Pi single board computers) was used as the operating system for its support and ease of use.

The audio interface, a MiniDSP MCH Streamer, was selected to acquire the 16 PDM audio channels. Despite its higher cost and limited accessibility, the device offered Linux compatibility and was supposedly designed for use with similar microphone arrays.

A generic USB camera (EMEET 1080p Webcam) was selected for this project due to its Linux compatibility and simplified interface compared to a MIPI camera connected to the Raspberry Pi's camera port. The camera operated at a resolution of 640 x 480, while supporting up to 1920 x 1080.

The touchscreen had a resolution of 1024 x 600, which provided sufficient screen space for controls and information given the camera's 640 x 480 resolution.

Microphone Array Design

The microphone array consisted of four quad-channel arrays arranged in a square to produce a 16-channel array of evenly spaced microphones with a spacing of 30mm. All boards shared a ground connection, clock input, and power input. A harness was constructed to connect these common connections to the MiniDSP MCH Streamer.

Each board utilized a buffer to clean up the clock signal before entering each microphone. Each board's four microphones were split into two pairs, which shared a PDM data line and were assigned to a "left" or "right" channel based on a pin that was pulled to ground or +1.8 volts. This configuration ensured each board possessed power, ground, clock in, data out 1, and data out 2 pins. See Figure 2 for a detailed schematic featuring decoupling capacitors and resistors, with values selected according to component datasheets.

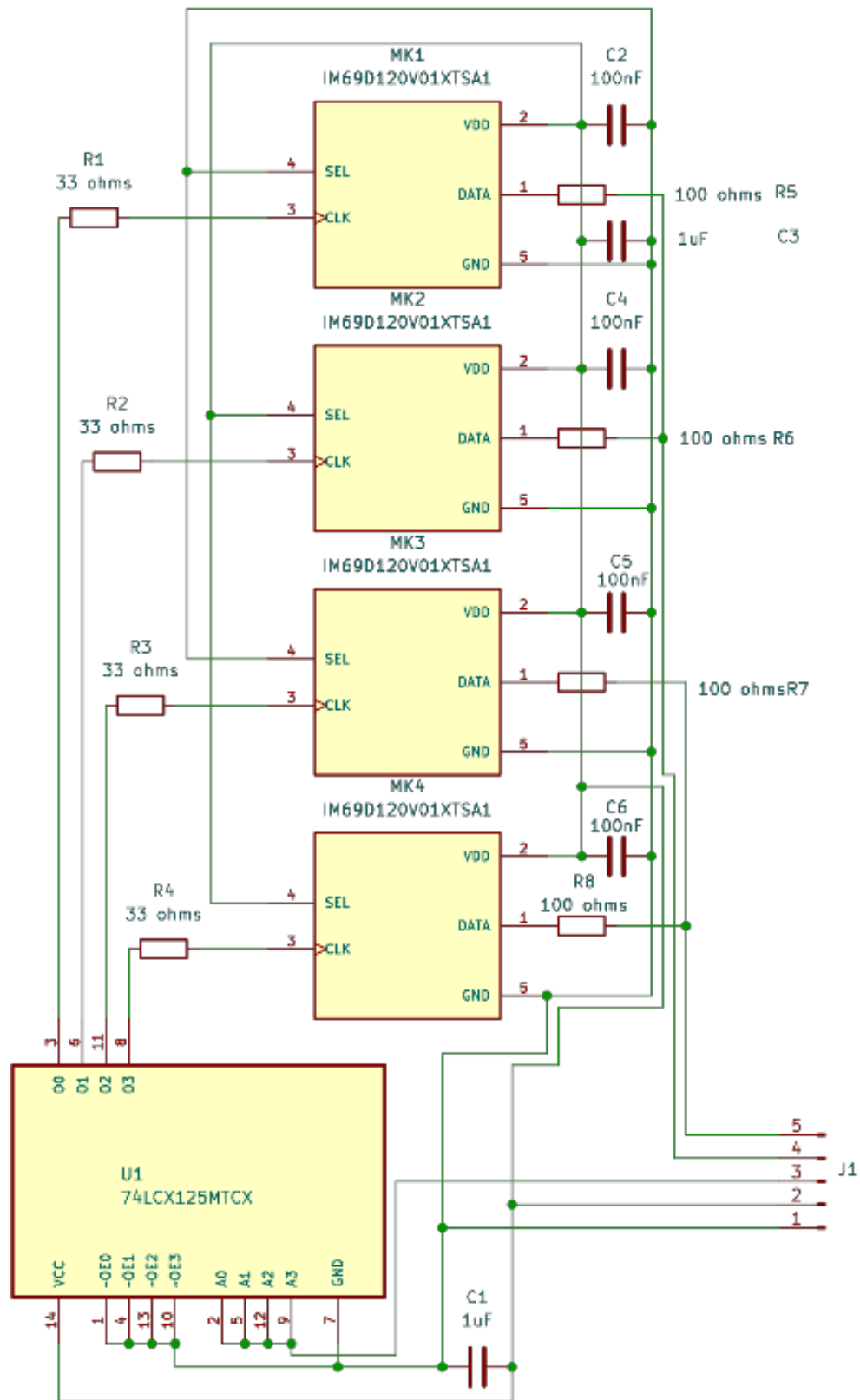


Figure 2: Quad-Channel, PDM Mic Array Schematic

Table 11 (located in the Appendix) contains a full bill of materials for each individual quad-channel array. The boards were assembled on a temperature-controlled hot plate, using low-temperature lead-free solder paste and no-clean flux. Custom boards were ordered through a low-volume PCB service, which included a stencil for applying the solder paste to the boards. A 3D-printed spacer was used to ensure accurate alignment and consistency for the assembly of the arrays.

The board layout was created based on the location of each microphone as they are the only element requiring a specific placement. Subsequently, a connector was added to the top of the board, and three holes were created for centering the array onto a 3D-printed faceplate. The equal-length traces for the PDM and clock signals are visible within Figure 3, which shows the rear side of the board. The University of Hartford Acoustics logo is prominently displayed on the front of the board (as visible in Figure 4) – primarily for aesthetic reasons.

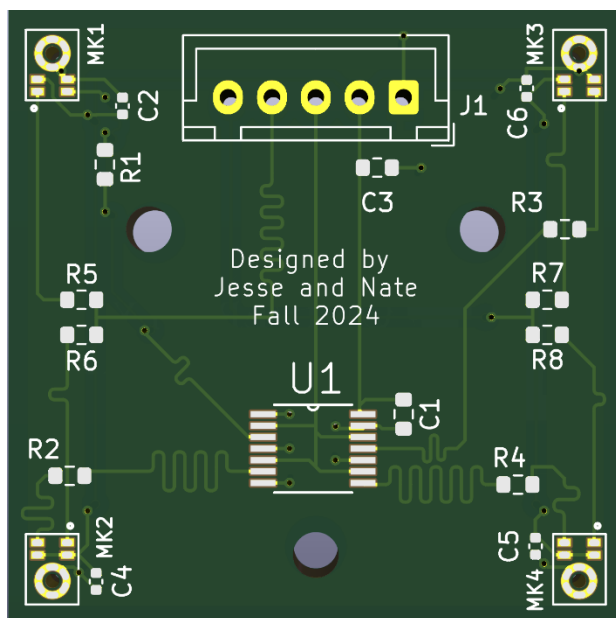


Figure 3: Microphone Array PCB Design, Rear



Figure 4: Microphone Array PCB Design, Front

Mechanical Component Design

The mechanical construction of the device utilizes 3D-printed components (printed from an Ender 3 3D printer) clamped together with through-hole bolts to provide structural integrity. Key components include a frame for touchscreen protection, a backing plate for mounting the touchscreen and Raspberry Pi, and a mounting plate for the camera and array boards. Subsequent paragraphs will detail the specifications and function of each component. All 3D parts were created in FreeCAD version 1.0.0.

Array Mounting Plate

The operational end of the device consists of a 3D-printed plate, detailed in Figure 5. This plate houses the microphone arrays, a camera mount, and features a series of precisely placed holes for each microphone (in red). The wells (in blue) provide a pocket for through-hole soldered connections, ensuring the microphone array boards sit flat. The entire plate mounts to the camera with four thru holes (in purple), each board has three mounting holes (in yellow), the camera has two mounting holes and a lens hole (in green).

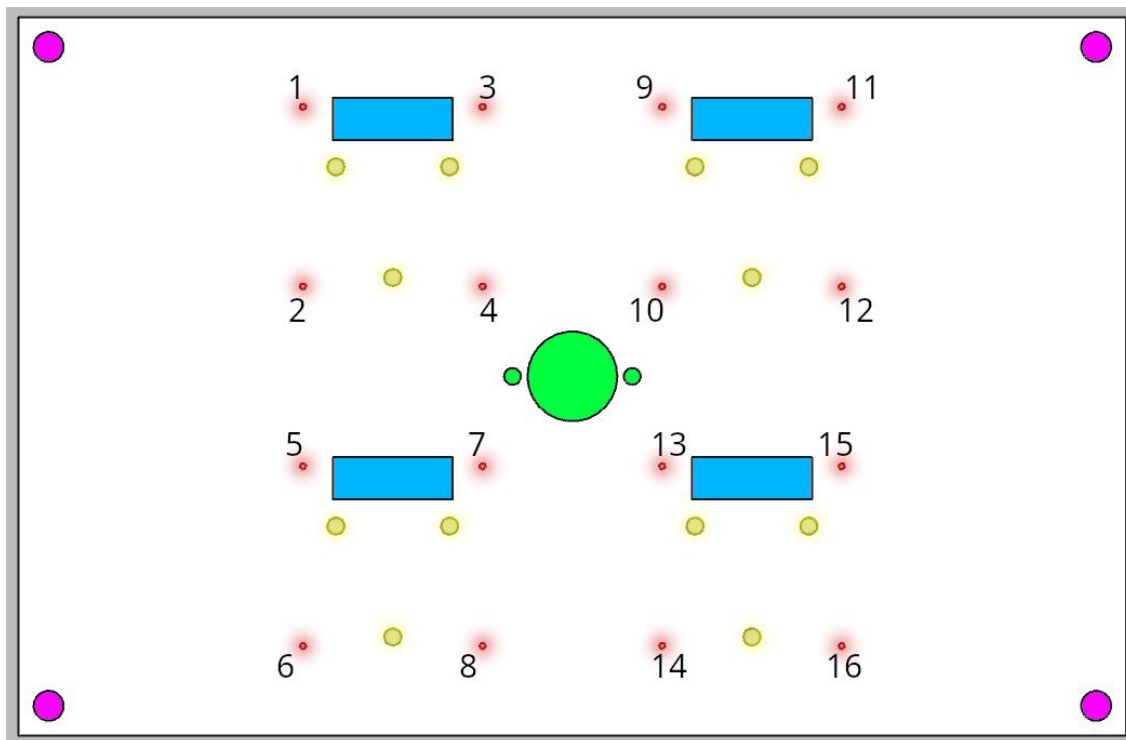


Figure 5: Mounting Plate for Camera and Arrays, Rear, with channels numbered

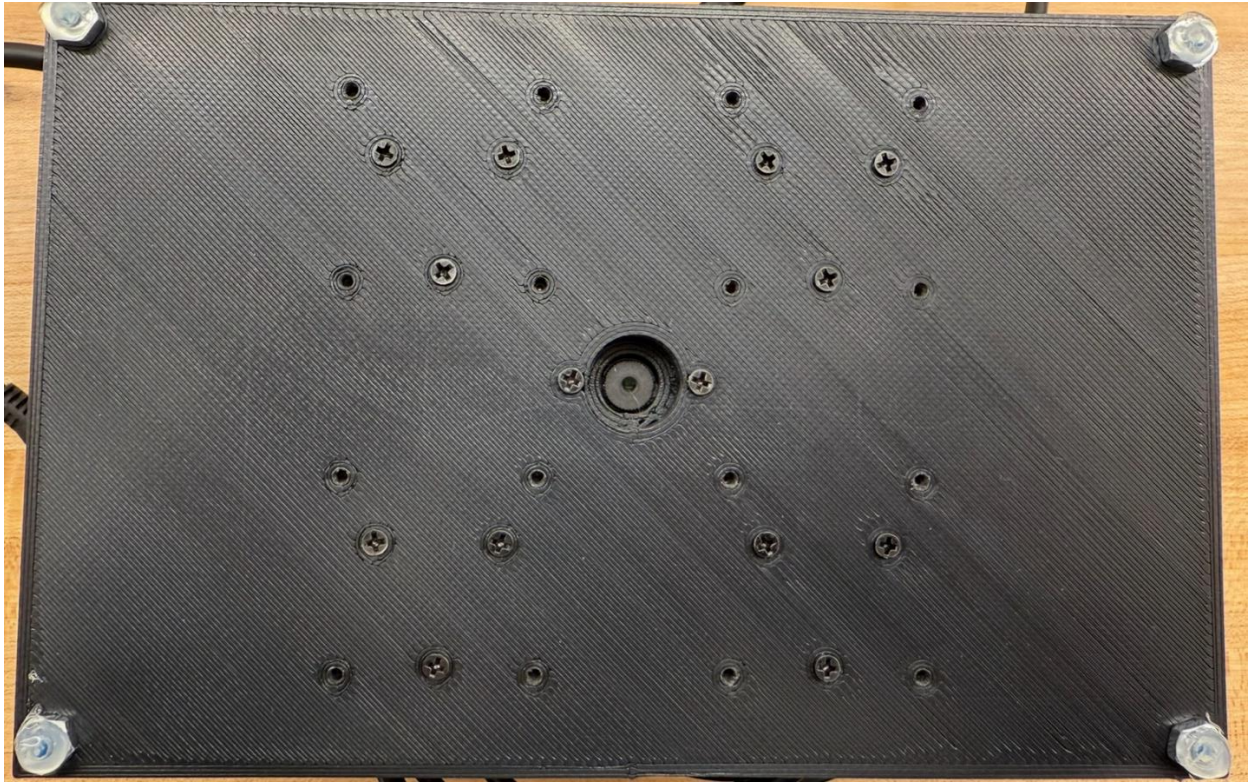


Figure 6: Mounting Plate for Camera and Arrays, Front View, Assembled

Microphone array boards were mounted to the plate, and then the Camera Adapter was installed, as shown fully assembled in Figure 7.

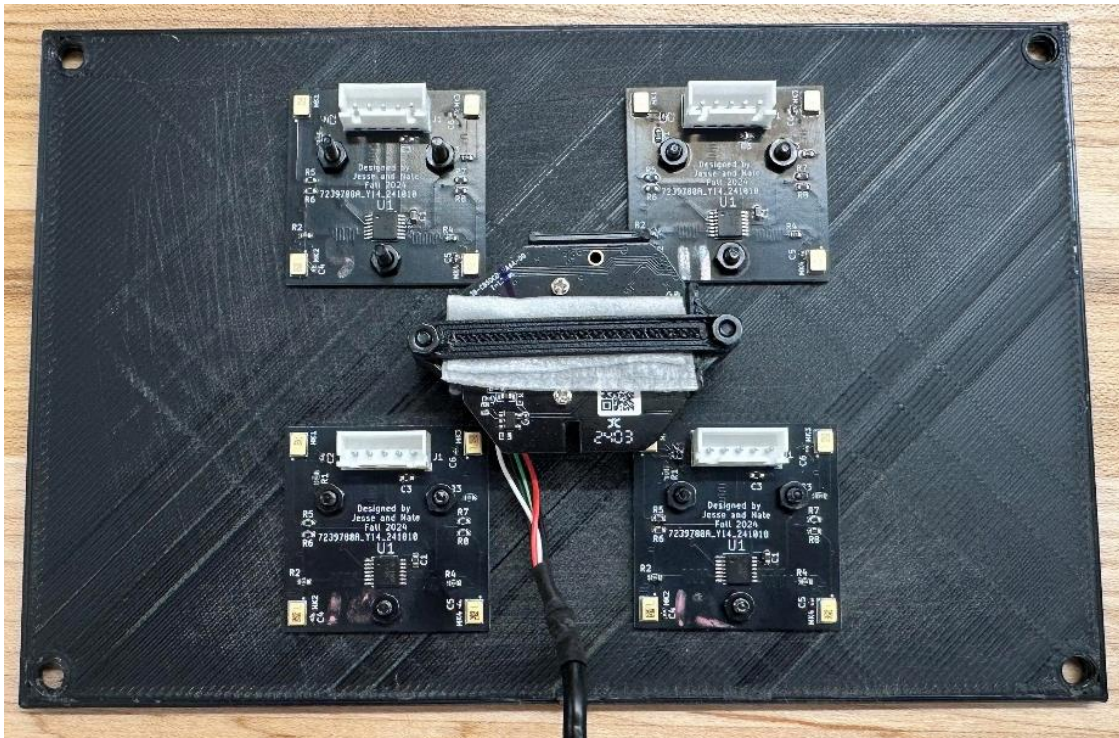


Figure 7: Assembled Mounting Plate, Rear

Camera Adapter

The camera adapter was designed for the exact dimensions of the webcam used. The webcam was disassembled, then the circuit board from the inside was used in the mount to minimize the space required for the camera. The adapter designed for the webcam used is shown below in Figure 8.

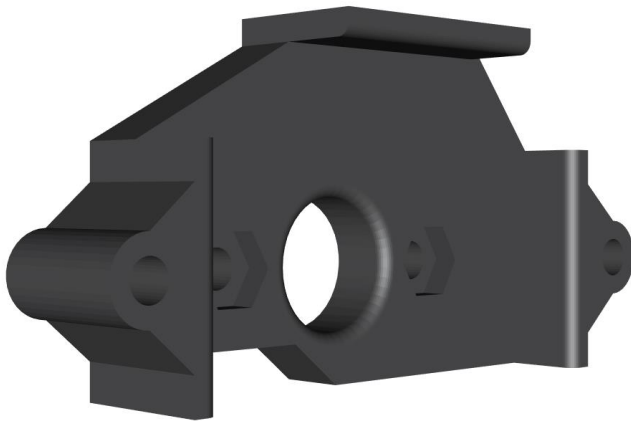


Figure 8: Camera Mounting Bracket

MCH Bracket

The MCH Streamer was mounted to the backside of the mounting plate using the bracket shown in Figure 9. The bracket utilizes the mounting holes on the MCH Streamer and existing bolts for mounting the array.

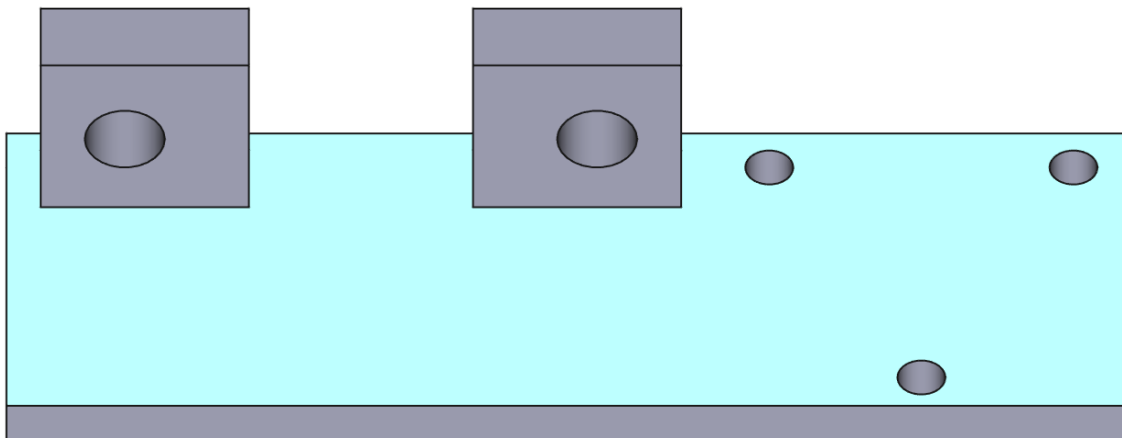
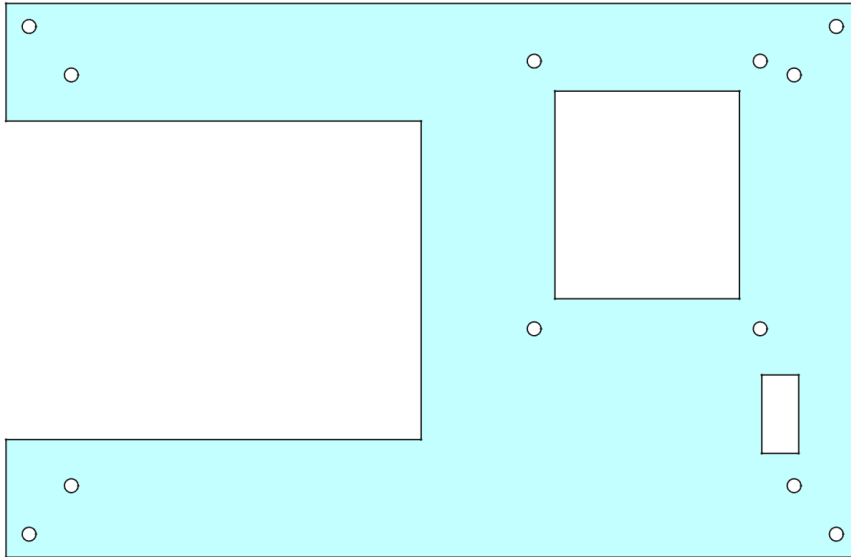


Figure 9: MCH Mounting Bracket

Screen Backplate

The screen backplate (Figure 10) mounts together the screen and the Raspberry Pi. This backplate contains thru holes for mounting the backplate inside of the camera, sandwiched between the screen bezel and the walls.

*Figure 10: Screen Backplate*

The screws for mounting the Raspberry Pi were inserted into the plate and bolted down (Figure 11), the screen was then attached to the plate as shown in Figure 12. The screws for mounting the Raspberry Pi were inserted into the plate and bolted down (Figure 11) ,the screen was then attached to the plate as shown in Figure 12.

*Figure 11: Mounting Plate with Screws for Pi*

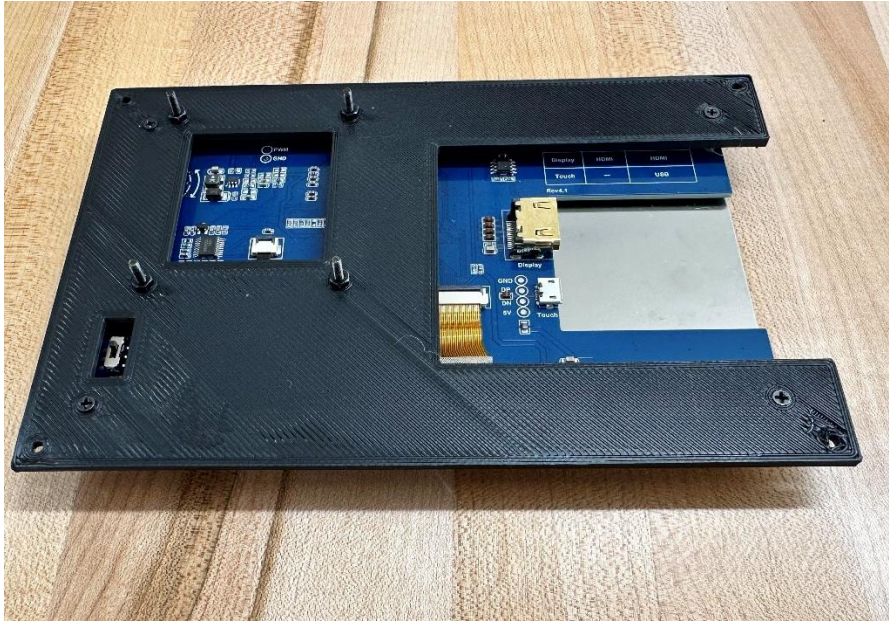


Figure 12: Mounting Plate with Screen Attached

Screen Bezel

The Screen Bezel exists to protect the exposed edges of the screen. This part was designed to fit the exact screen used in this project, as shown in Figure 13. It first has bolts run through it (Figure 14), then the mounting plate with screen attached is placed into the bezel (Figure 15).



Figure 13: Screen Bezel



Figure 14: Screen Bezel with Bolts



Figure 15: Screen Bezel with Screen Backplate installed



Figure 16: Screen Bezel assembled, alternate view

Power and Cooling Considerations

The Raspberry Pi requires a USB-C power source capable of providing 5 volts at 5 amps. This is a concern for most USB-C power supplies and battery packs as most do not provide enough current. A suitable portable power source has not been determined yet, but a custom solution has been speculated in

Future Development Paths.

The Raspberry Pi 5 requires a heatsink to operate without thermally throttling. The official active cooler is an affordable and simple solution but utilizes a small fan. The fans' noise and vibration may limit the minimum levels the camera can detect. It is suspected that the Pi will operate without throttling with the fan disconnected, but more investigation into the thermal performance of the heatsink when limited to free convection is required.

Software Design

The following section breaks down the flow of signal as it enters the camera and traces all steps of processing the audio and video.

Audio

Audio processing is performed in three main steps. First the incoming data is beamformed using the precalculated steering vector (weights), then the beamformed data is passed through a Fast Fourier Transform (FFT), lastly the desired frequency bin is converted to decibels full scale (dBfs) and sent to the video processing portion of the code.

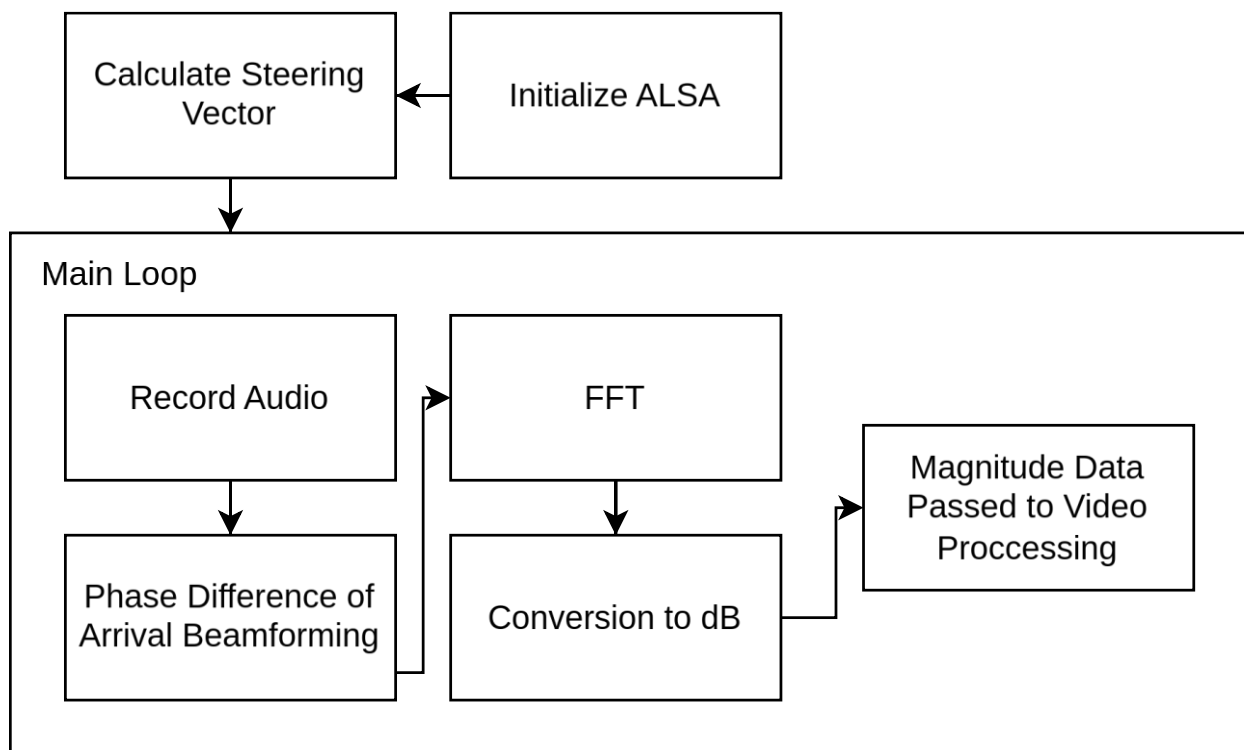


Figure 17: Audio Processing Overview

Audio Recording

Audio is recorded using the library ALSA. First, parameters are set such as the frame size, sample rate, audio device, and data type for the incoming audio data. Audio is continuously recorded

on a separate thread into a ring buffer to ensure a constant stream of audio. Dispatching a separate thread that only runs the audio recording ensures that any slowdowns on the main thread don't stop ALSA from capturing a frame of audio on time.

The audio is received in the format of a 32-bit little endian signed integer from the audio interface that is interleaved. This means that the data comes in shuffled in a particular manner. It is preferable for the audio to be stored where all 1024 data points for one microphone are in a row, followed by the next microphone, and so on. Additionally, the order that the audio interface sees the microphones is not necessarily in the correct order for processing so that must be addressed at this stage. Additionally, the incoming data must be converted to a float.

The ring buffer consists of two (2) frames of audio of dimensions $4 \times 4 \times 1024$. To record a frame of audio, first buffer 2 is swapped with buffer 1. Then, data is written into buffer 2. To read data from the ring buffer, it must be read from buffer 1. The purpose of the ring buffer is to ensure that there is always a frame of audio ready to be read by the audio processing portion of the code.

Beamforming

Most often, beamforming is done in the context of radio frequency engineering or ultrasound applications. Due to the wave nature of sound and electromagnetic waves, the same concepts apply to sound. The output of beamforming is often described as array factor (AF). “Array factor represents the response of an array of isotropic elements” [4]. In the case of an acoustic camera, the input is from a Uniform Rectangular Array (URA) of microphones.

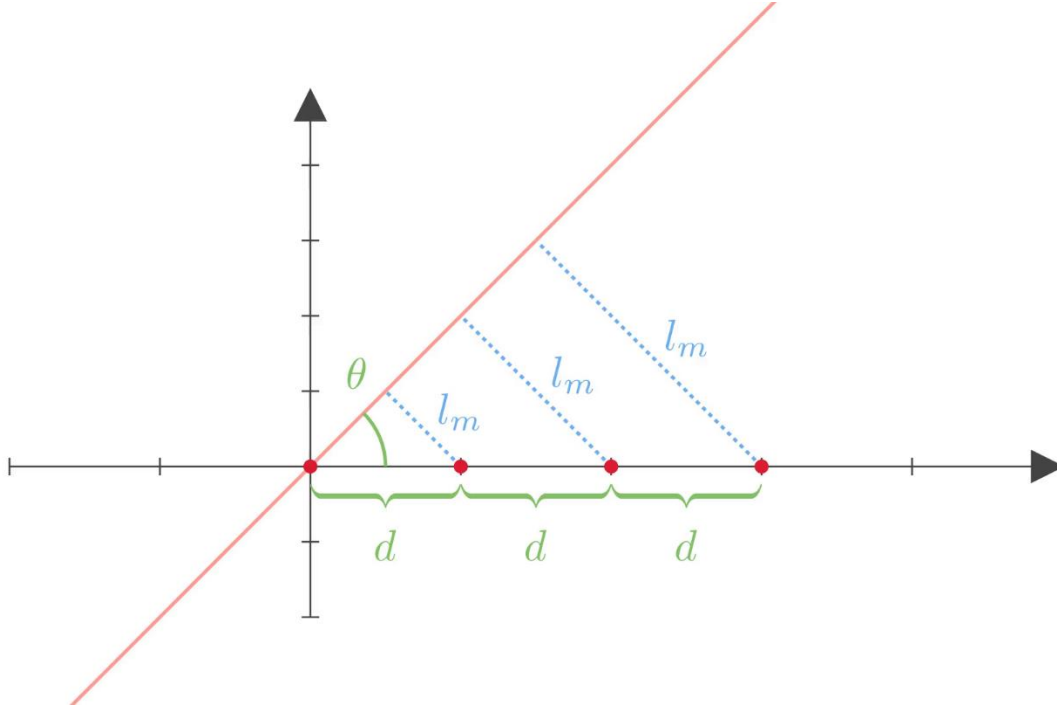


Figure 18: URA of Microphones Demonstration

Figure 18 shows a ULA with a plane wave of sound approaching at some angle θ . The wave is travelling at the speed of sound in the direction normal to its surface. The plane wave impacts array elements at a slightly different time. With this difference in time also comes a difference in the phase as only a single frequency is considered.

Other patterns of microphone arrays may be chosen that employ uneven spacing between microphones, but this project chose to use a URA for simplicity and ease of manufacture. The fundamental equation that describes the array factor at a given azimuth and is shown in Equation 2.

$$AF(\theta) = \sum_{m=0}^{M-1} x_m e^{-idkmsin(\theta)}$$

Equation 1 [5]

The elevation angle ϕ can be substituted for θ in Equation 2 and combined with it to obtain the total AF for the case of a URA (Equation 2). Together, θ and ϕ represent a new coordinate system where θ can be plotted along the x-axis and ϕ can be plotted on the y-axis for the video to display. It should be noted that AF is a complex number with real and imaginary components.

$$AF(\theta, \phi) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{mn} e^{-idk(msin(\theta)+nsin(\phi))}$$

m = index of microphone in horizontal axis

n = index of microphone in vertical axis

x_{mn} = incoming signal from microphone at index (m, n)

θ = azimuth angle

ϕ = elevation angle

$$i = \sqrt{-1}$$

d = spacing between each microphone

k = wave number

Equation 2 [5]

FFT

A Fast Fourier Transform (FFT) is performed on the data from the microphones after the beamforming algorithm is applied. Several one-dimensional FFT's are performed on the incoming data, one for each pair of angles θ and ϕ , using the fftw3 library with a size of 1024 samples. The FFT library does not normalize the output of the FFT so each element is divided by the size of the FFT (1024 samples).

Final Processing

Incoming AF data is converted into amplitude, then into dBfs only for the frequency bins in which beamforming was applied using Equation 3. It should be noted that the signal inside the logarithm is divided by the maximum possible signal value, but in this case the value is 1 so it does not appear in the equation. The range of the data output is -100 to 0 where 0 represents the maximum possible signal value.

$$AF [dBfs] = 10 \log_{10}(\sqrt{real(AF)^2 + imag(AF)^2})$$

Equation 3

Video Processing

Video processing is performed on a frame-by-frame basis, utilizing raw frames from the camera and magnitude data from the audio processing. As illustrated in the overview block diagram in

Figure 19, the script initiates with initialization and configuration loading, followed by the commencement of the frame capture thread. Video frames are merged with a heatmap generated from the magnitude data, following processing based on user-defined settings. A color bar and marker for the maximum magnitude are then drawn on the merged frame. This processed frame is rendered in a window with controls using ImGui, and settings are updated based on user input. This process is repeated with each new frame of magnitude data.

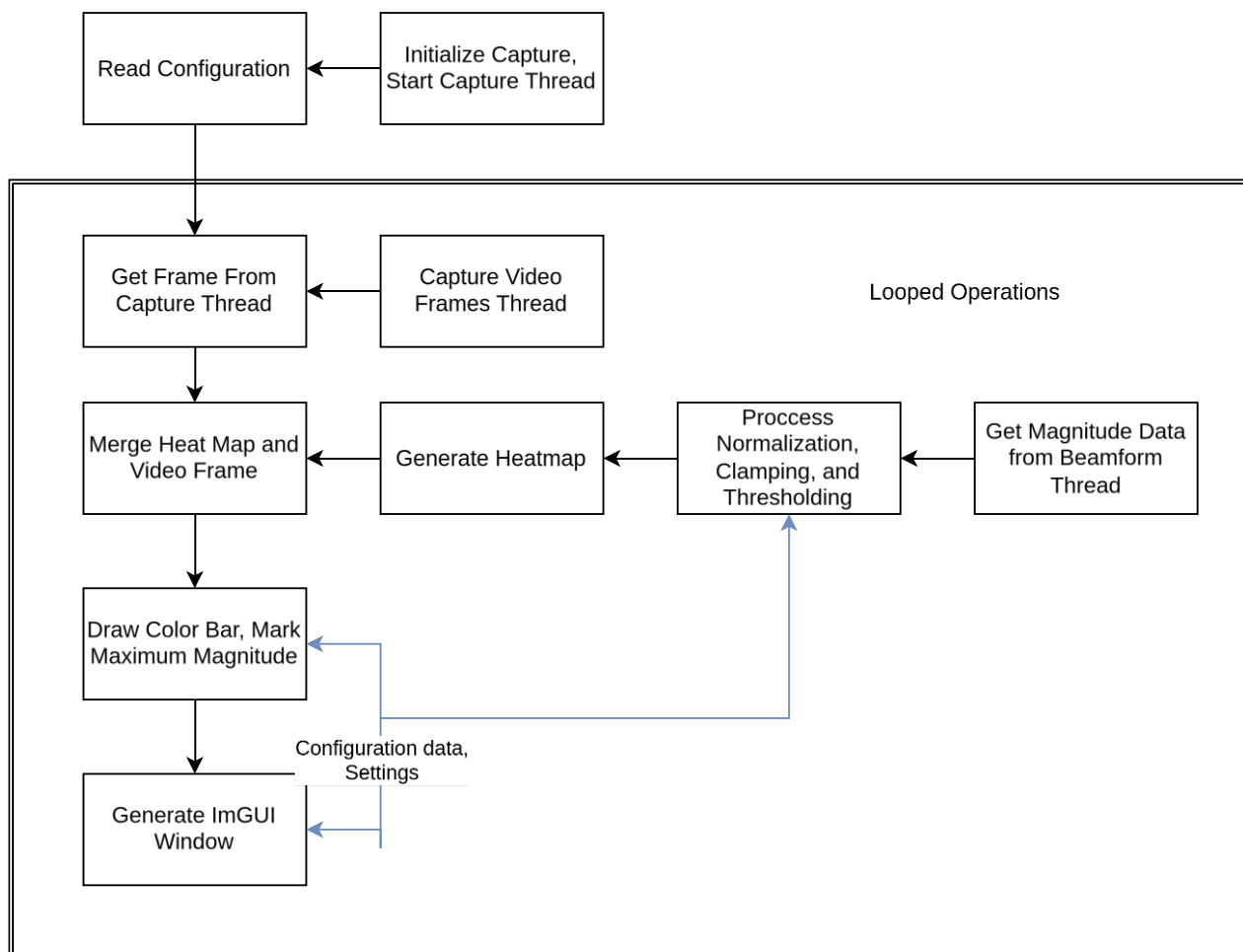


Figure 19: Video Processing Overview

GUI Overview and Operational Instructions

The Acoustic Camera application operates out of one main window, as shown in Figure 20. This main window features three components: the video display, info menu (Figure 21), and a panel on the right hand side that can contain the controls menu (Figure 22) or the options menu (Figure 23). This interface will be overhauled in future versions to simplify user experience.

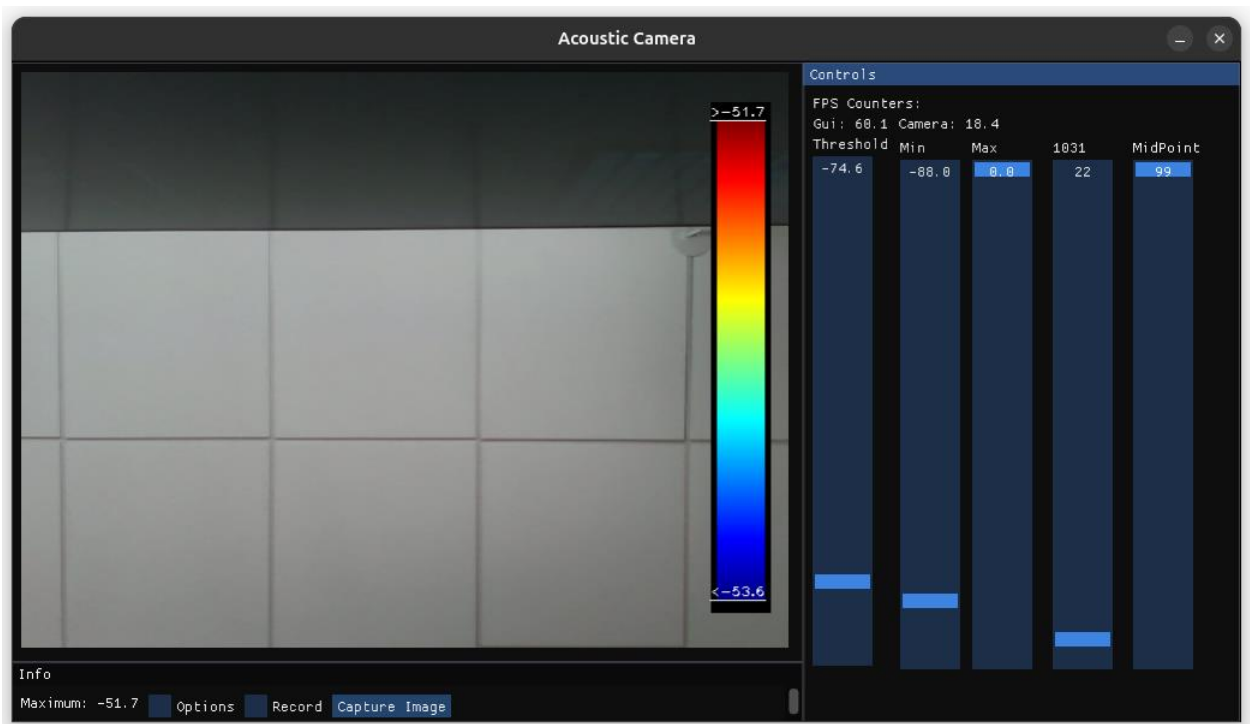


Figure 20: Acoustic Camera GUI

The info menu (Figure 21) features the following: a readout of the maximum level displayed, a check box to switch the right-hand side from the controls menu to the options menu, a check box to enable video recording and a push button to capture a still image. See options menu section for details on selecting a destination for captured images. Captured images are named “image_YYYYMMDDHHMMSS.png” where the date and time is the local time of capture.

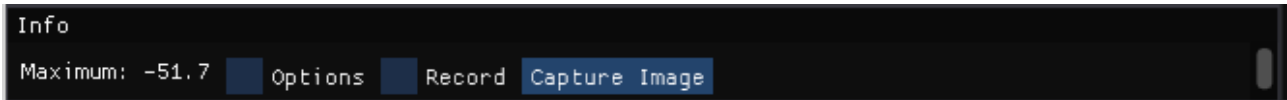


Figure 21: Acoustic Camera GUI: Info Menu, cropped and enlarged for legibility

The controls menu shown in Figure 22 contains the current FPS rate of different components of the application, and five sliders for controlling the application. See Table 2 for information regarding the function of each slider. Sliders can be adjusted for filtering and processing data using the touch screen of the device for easy on-the-fly adjustment.

Table 2: Sliders and their Functions

Slider Name	Slider Function
Threshold	Sets value at which any value below is transparent in heatmap
Min	Clamps all values below or equal to this value
Max	Clamps all values above or equal to this value
Frequency	Displays the currently center frequency of the selected bin
Mid-Point	Shifts the appearance of the heatmap by setting the percentage of data points at the midpoint

The options menu, in Figure 23, features checkboxes for various functions, which are outlined in Table 3. The menu features one button, which is outlined in Table 4. The alpha slider determines how opaque the heatmap is.

Table 3: Options Menu Checkboxes and their Functions

Checkbox Name	Checkbox Function (when checked)
Mark Max Mag	Marks the maximum magnitude direction
Color Scale	Displays a color scale over the image
Heat Map	Applies a heat map to the image
Data Clamp	Activates the clamping sliders
Threshold	Activates the thresholding slider
AutoSave Config	Automatically saves any changes so app retains settings and controls

Table 4: Options Menu Buttons and their Functions

Button Name	Button Description
"Select Save Directory"	Opens a menu when pressed for navigating to the desired working directory

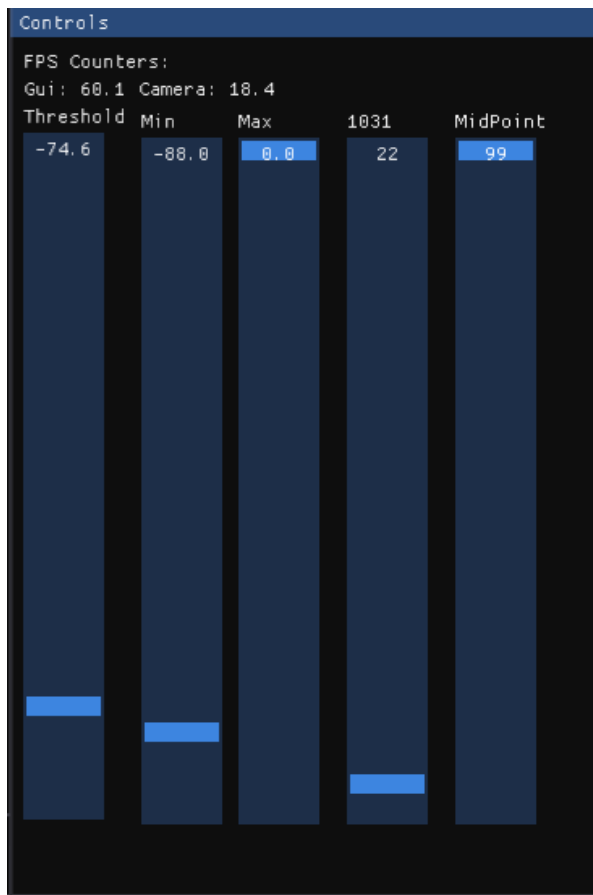


Figure 22: Acoustic Camera GUI: Controls Menu

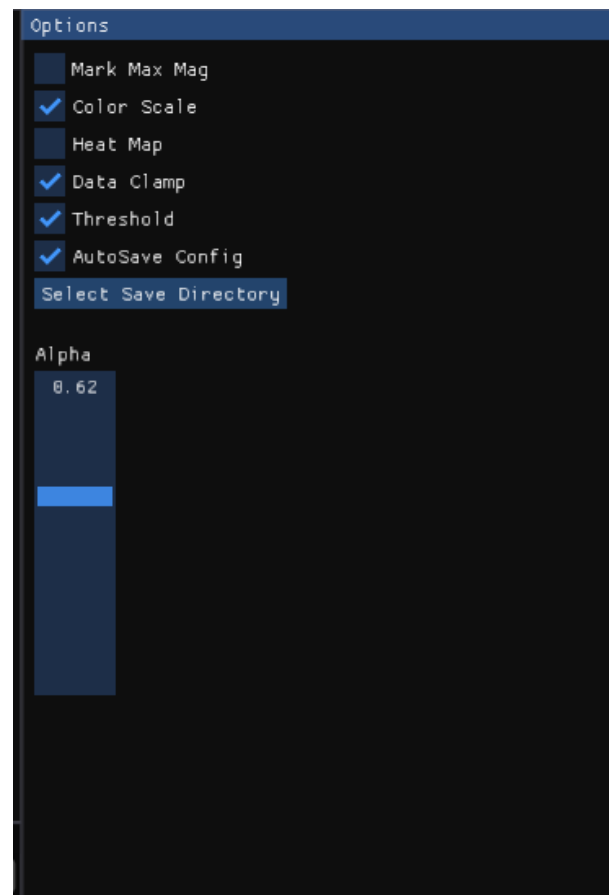


Figure 23: Acoustic Camera GUI: Options Menu

Software and Prerequisite Library Installation

The Acoustic Camera software requires the installation of numerous prerequisite libraries and packages. These are available through Git [6] and/or Apt [7]. The following sections detail the process for installing these dependencies. Code development for the Acoustic Camera project occurred in Visual Studio Code, utilizing both a Raspberry Pi and an Intel-based laptop running Ubuntu. Detailed specifications are listed in section 0.

Dependencies

ImGui Usage

To facilitate user interaction and control, the project incorporates Dear ImGui, an immediate mode graphical user interface library. All required ImGui source files are included within the project's distribution. This project utilizes ImGui under its licensing terms and does not represent a contribution to the ImGui codebase; it is used solely for the purpose of creating the acoustic camera's user interface. [<https://github.com/ocornut/imgui>]

Other Required Dependencies

Prior to installing any dependencies, the Raspberry Pi was updated using “sudo apt-get update” and “sudo apt-get upgrade”. Table 5 details the commands used to install dependencies for various packages. These dependencies must be installed before OpenCV and the Acoustic Camera software. Visual Studio Code is recommended for modifying the Acoustic Camera project.

Table 5: Required Dependencies

Required For:	Command to Install
OpenCV	sudo apt-get install build-essential cmake git pkg-config libjpeg-dev libtiff-dev libpng-dev libavcodec-dev libavformat-dev libswscale-dev libv4l-dev libxvidcore-dev libx264-dev libfontconfig1-dev libcairo2-dev libgdk-pixbuf2.0-dev libpango1.0-dev libgtk2.0-dev libgtk-3-dev libatlas-base-dev gfortran python3-dev python3-numpy libasound2-dev
Acoustic Camera	sudo apt-get install libsdl2-dev libglew-dev libfftw3-dev

OpenCV Installation

OpenCV (Open Source Computer Vision Library) is an open-source computer vision and machine learning library designed for real-time image and video processing. It provides a wide range of tools and algorithms for tasks such as object detection, image segmentation, feature extraction, and motion tracking. Written in C++ with bindings for Python and other languages, OpenCV is optimized for performance and widely used in applications ranging from robotics and augmented reality to medical imaging and autonomous systems [8].

The usage in this project is for capturing video frames, processing the frames (including generating the heatmap, and overlaying information over the output frame) and filtering the video frames. OpenCV was installed using the following procedure in Table 6. Note: the “make -j4” command may take significant time to complete.

Table 6: OpenCV Installation Commands

Step	Command
Clone OpenCV from Git	<code>git clone https://github.com/opencv/opencv.git && git clone https://github.com/opencv/opencv_contrib.git</code>
Create and open build directory	<code>cd ~/opencv && mkdir build && cd build</code>
Configure the build process	<code>cmake -D CMAKE_BUILD_TYPE=RELEASE \</code> <code>-D CMAKE_INSTALL_PREFIX=/usr/local \</code> <code>-D OPENCV_EXTRA_MODULES_PATH=~/opencv_contrib/modules \</code> <code>-D BUILD_TESTS=OFF \</code> <code>-D INSTALL_PYTHON_EXAMPLES=OFF \</code> <code>-D BUILD_EXAMPLES=OFF \</code> <code>-D BUILD_SHARED_LIBS=ON \</code> <code>-D OPENCV_GENERATE_PKGCONFIG=ON \</code> <code>-D WITH_V4L=ON \</code> <code>-D WITH_LIBV4L=ON \</code> <code>-D WITH_FFMPEG=ON \</code> <code>-D WITH_GSTREAMER=ON \</code> <code>..</code>
Compile OpenCV using four cores	<code>make -j4</code>
Install OpenCV	<code>sudo make install</code>
Update dynamic linker bindings	<code>sudo ldconfig</code>

Compatibility

Software compatibility is currently largely untested. However, the use of the ALSA sound driver currently restricts support to Linux operating systems. Future versions may overcome this limitation by transitioning away from ALSA. OpenCV is available for a wide range of

architectures and operating systems, while all other prerequisites are specifically designed for these libraries. The camera's functionality is limited by the compatibility of OpenCV. Plug-and-play, Linux-compatible USB webcams are expected to be largely compatible with Acoustic Camera software.

The specifications for the Raspberry Pi 5 are detailed in Table 7. The development process took place on the Raspberry Pi, alongside one laptop and one virtual machine. Specifications are listed in Table 8 and Table 9. The performance of the acoustic camera software is significantly better on the laptop than the Raspberry Pi, allowing debug and testing to be more efficient.

Table 7: Raspberry Pi 5 Specifications

Device	Raspberry Pi 5
Operating System	Raspberry Pi OS bookworm (64-bit)
Processor	Broadcom BCM2712
Memory	8 GB LPDDR4X-4267
Storage	64 GB SanDisk Extreme Plus Micro SD Card

Table 8: Development Laptop Specifications

Device	Framework 13
Operating System	Ubuntu 24.10
Processor	Intel i7-1185G7
Memory	32 GB DDR4-3200
Storage	Crucial T500 – 1TB NVMe Drive

Table 9: Virtual Machine for Development Specifications

Device	Virtual Machine (VMWare)
Operating System	Raspberry Pi OS bullseye i386
Processor	1 CPU 4 Cores
Memory	4GB Ram
Storage	20 GB Virtual Disk

Results

The camera was evaluated in various spaces to test its functionality. The resulting performance was reviewed qualitatively, as time became a large constraint in testing.

Pink noise played through an iPhone 16 Pro was used as a source in the following tests. The testing locations include an anechoic chamber, reverberation room, and a lecture hall.

The color scale in the following images is not entirely accurate, as the only correct values are the minimum and maximum, at the bottom and top of the scale, respectively.

Frames captured from the video recordings only demonstrate performance of the camera at specific points in time, and do not accurately represent noise and variation in the output of the camera.

Figures are labeled with TL for Top Left, TR for Top Right, BL for Bottom Left, and BR for bottom right.

Paul S. Veneklasen Research Foundation Anechoic Chamber

The following images, Figure 24, Figure 25, Figure 26, and Figure 27 are four frames of a video recording where the source was moved to four corners of the screen space and the camera was held steady. This was recorded in the Paul S. Veneklasen Research Foundation Anechoic chamber at the University of Hartford.

.



Figure 24: Anechoic Chamber Test, TL



Figure 25: Anechoic Chamber Test, TR



Figure 26: Anechoic Chamber Test, BL



Figure 27: Anechoic Chamber Test, BR

The camera marked a location of maximum magnitude near the source in all four corners of the test, but with a consistent error in location. The consistent error in location is suspected to be due to the lack of lens distortion correction in the current code. For simplicity of development and testing, the cameras edge angles are configured to be the end angles for beamforming, and the angular coordinates are linearly mapped to the screen space. This method is not ideal in terms of accuracy, but rather a predictable way to prove the functionality of the device.

Paul S. Veneklasen Research Foundation Reverberation Room

The following images, Figure 28, Figure 29, Figure 30, and Figure 31 are four frames of a video recording where the source was moved to four corners of the screen space and the camera was held steady. This was recorded in the Paul S. Veneklasen Research Foundation Reverberation Room at the University of Hartford.



Figure 28: Reverberation Room Test, TL



Figure 29: Reverberation Room Test, TR



Figure 30: Reverberation Room Test, BL



Figure 31: Reverberation Room Test, BR

All four test locations had a resulting marked maximum magnitude in the general area of the source. These frames were not as aligned or consistent as in the anechoic chamber but follow the general location of the source.

Dana Hall Mali 1 Lecture Hall

The camera was evaluated in a ~70 seat lecture hall at the University of Hartford. The source was held along the rear of the lecture hall, while the camera was held in the front. Sequential frames are provided in Figure 32 through Figure 37.



Figure 32: Lecture Hall, Longer Distance, Frame 1



Figure 33: Lecture Hall, Longer Distance, Frame 2

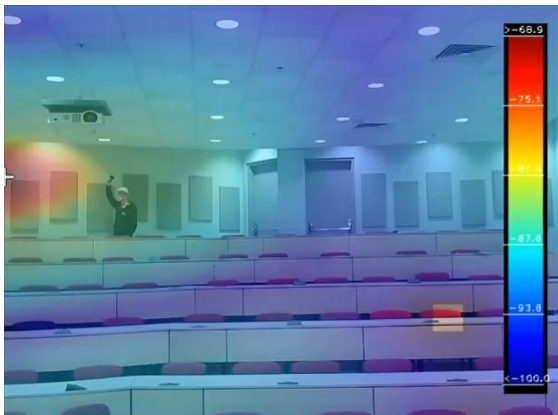


Figure 34: Lecture Hall, Longer Distance, Frame 3

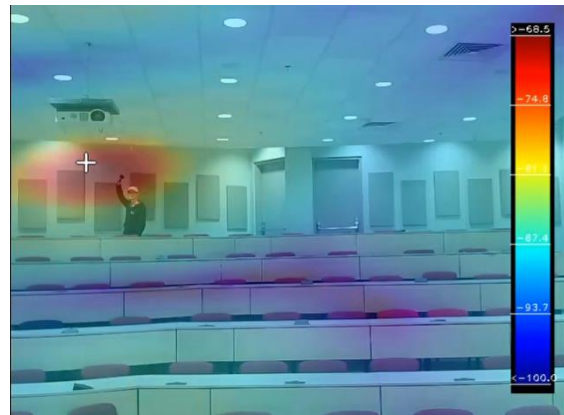


Figure 35: Lecture Hall, Longer Distance, Frame 4

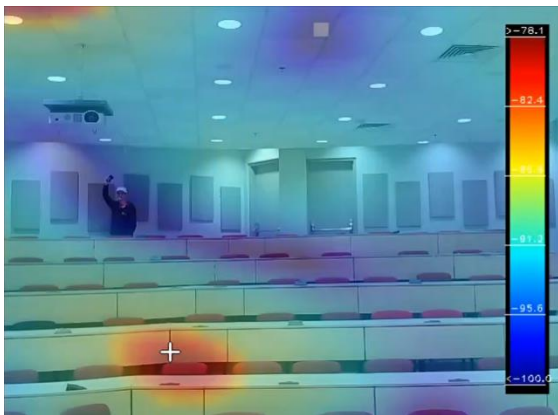


Figure 36: Lecture Hall, Longer Distance, Frame 5

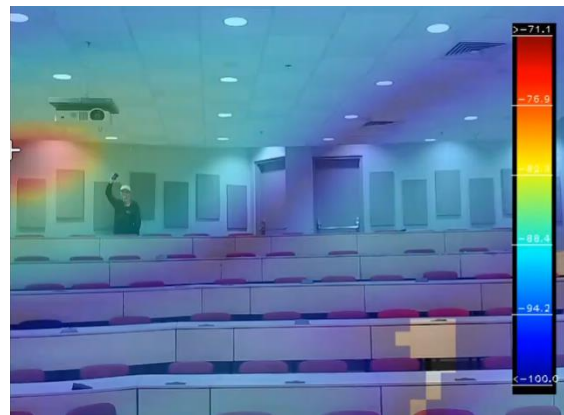


Figure 37: Lecture Hall, Longer Distance, Frame 6

The locations of marked maximum magnitude appeared either near the source, or in the path of a potential direct reflection from the source to the camera. The points marked not directly on the source appear to follow the path of a direct reflection, specifically in Figure 32 and Figure 36.

Cost

The cost of the open-source acoustic camera is significantly less than that of commercially available cameras. Table 10 outlines the main components used in this design, and their associated cost.

Table 10: Component Costs at Time of Purchase (2025)

Item	Cost
Raspberry Pi 5 Kit	\$ 119.99
MCH Streamer	\$ 95.00
Touchscreen	\$ 52.99
SMD Components	\$ 18.75
EMEET Webcam	\$ 16.99
Pi 5 Active Cooler	\$ 8.15
Stencil for SMD Soldering	\$ 7.11
Custom Array Board (4)	\$ 1.48
Total Cost:	\$ 320.46

These costs do not include PLA filament, solder, soldering flux, soldering paste, assembly hardware, or cables. Taxes, tariffs, and shipping fees are also not included, and are subject to significant change based on a variety of factors. The costs listed in the table reflect prices at their time of purchase in Fall 2024 and Spring 2025. Required tools for assembly are not included in this list, either. It is recommended to have the microphone array boards assembled by the board manufacturer, as many tools and difficulties arise with soldering these boards in a DIY fashion. If attempting to DIY solder the boards, low temperature solder paste is recommended, and the cost of the stencil will be reduced by specifying the size of the stencil.

Discussion and Conclusions

Performance

Current performance on the Raspberry Pi is limited to around 20 frames per second (fps). Many animations in popular media are animated at 12 fps, so this can be seen as a lower bound for an acceptable frame rate as this is an industry standard fps. The upper bound for frame rate is limited by the audio sample rate and the size of the FFT. This upper bound is calculated as per Equation 4.

$$fps_{MAX} = \frac{f_s}{N} = \frac{48000 \text{ Hz}}{1024 \text{ samples}} = 46.875 \text{ fps}$$

Equation 4

Precision and Accuracy

Currently, there are many contributing factors limiting the precision of the acoustic camera. Most of these factors can be mitigated with further development.

Azimuth and Elevation Resolution

For the testing outlined in this report, the camera was configured to sweep across -30 to 30 degrees in increments of 2 degrees for azimuth, and -18 to 18 degrees in 1-degree steps for elevation, providing an overall angular resolution of 30 by 36 data points. With further optimization of beamforming performance, this resolution could be increased to be a smaller step size. Selecting a wider-angle camera could also improve the resolution as the density of angles in the screen space would be improved.

Lens Distortion

In the camera's current development, lens distortion is not accounted for, and the location of angular coordinates is linearly approximated from edge to edge of the camera. This is potentially the cause of the constant locational error visible in the Paul S. Veneklasen Research Foundation Anechoic Chamber testing. The error existing in the marked location versus the phone is likely due to the differences in mapped angular locations versus the actual location of objects in the camera's view.

Side Lobes

Side lobes because of beamforming are also visible in the results as the patterns present not near the sound source. There is no mitigation of these currently present in the software.

Design Oversights

There have been several “teaching moments” throughout the course of the design and development that are important to make note of. These topics are instances in which hindsight greatly impacts perspective on previous design decisions.

Hardware Design Considerations

The microphone arrays and audio interface represent a suitable choice, though a potentially better alternative exists. Given the 16 channels of PDM audio, the MCH Streamer is the most sensible solution. However, an alternative is possible.

Software Oversights

Throughout development, many software features were not given development time that should have been addressed early. For example, audio recording was not implemented until very late in the project cycle, so all efficacy testing of the array had to be done with the physical hardware, limiting the development time available.

Although C++ is a more performant programming language than Python or MATLAB, either of the latter options have built in functions to handle beamforming, so the efficacy of the microphone array could have been tested very early on. Performance may have been lacking compared to a scratch-made C++ version but there could have been a save in development time trying the Python or MATLAB functions in the beginning.

Array Assembly Difficulties

The microphone array boards, and the corresponding surface mount components were ordered and assembled on a hot plate. This method cannot be recommended. Low temperature solder paste was applied to the boards using a stencil and a credit card. The stencil was then carefully removed, and components were placed by hand onto the paste. The boards were then placed onto a 210-degree Celsius hotplate for two minutes, then removed and allowed to cool. The success rate of this method resulted in 8/15 assembled boards functioning correctly.

Many errors were made in this process, including but not limited to the time and temperature profiles not matching the data sheet for the microphones and low-quality solder paste usage.

Future iterations will be assembled from an assembly service, avoiding these issues. An individual with more surface mount soldering experience may have better luck assembling at home.

Future Development Paths

Microphone Array Redesign

As proposed in previous sections, the microphone arrays will be overhauled using TDM (Time Division Multiplexing) instead of PDM audio, a microcontroller for conversion of digital audio into serial data, and serial to transmit the data into the device. This redesign should minimize cost and complexity, allowing for flexibility in the microcontroller used.

A redesign, utilizing TDM microphones connected to a microcontroller such as a Teensy 4.1, is one potential solution. The switch from PDM to TDM in the design is to allow all 16 channels to be connected to the microcontroller through one input. This would involve programming the Teensy to convert and transmit the TDM data as serial data, potentially via USB or SPI. This approach offers a lower cost and greater operational customization compared to the closed-source firmware of the MCH Streamer, while also reducing complexity and improving compatibility by eliminating the need for audio drivers. This hardware design is currently in its earliest stages of theoretical development but may represent a simpler and more economical solution.

This proposed method using TDM microphones is not without its concerns. Maintaining accuracy of timing among channels, bandwidth limitations of serial interfaces, and overall processing power of a microcontroller may prove to be significant challenges with this approach. Further investigation is required to determine if this is possible, and a better solution.

Improved Beamforming

Several methods are available in the future for improving the beamforming portion of the code. One such method is GPU acceleration. It may be used to improve the performance of the beamforming algorithm as many different frequencies can be calculated in parallel using compute shaders. Performance of GPU acceleration is to be evaluated in the future. GPU acceleration was not explored during development as to avoid premature optimization, instead effort was put into proving the concept, accepting limitations that arise.

There are several different methods of beamforming that were not explored in this project that could prove to be more accurate or quicker. Other methods may be explored in the future with the knowledge gained throughout development to help assure that beamforming is possible with the hardware created. It is now evident that the hardware is sufficient to perform beamforming and any issues that arise would be with the software.

Compatibility

Through moving away from the MCH Streamer and the ALSA driver, compatibility with Windows and Mac based devices will be achievable. This would allow for x86 based single board computers and laptops to be used for the processing of the camera.

Acoustic Camera Module

The redesign of the microphone array and the improved compatibility will pave way for a version of the camera that is just the microphone array, video camera, and microcontroller. This version will be a module that connects to an existing computer or device through USB. By moving all processing and display into the users' existing computer, the cost is greatly reduced for those who want a cheap and simple solution. The convenience of a standalone device is lost but made up for in cost for certain applications.

Audio and Video Resolution

The device is currently greatly limited in angular, video, and frequency resolution. Controls for adjusting these values on the fly will be added so a user can dial in what the user needs. Adaptive resolution is another potential feature, where the Acoustic-Camera software can focus on regions of maximum magnitude automatically.

Power Supply

The device is currently only operational with a USB C power supply that can deliver 25 watts at 5 volts and has no internal battery. A solution could be using a battery pack capable of a higher wattage (65 watts may be a target range due to inefficiencies), a buck converter capable of outputting 5 volts at 5 amps, and then connecting to the 5 volt and ground pins on the GPIO header of the Pi. This method has not been assembled and tested yet but is the next step in making this a portable device. Limitations are the lack of a proper shutdown in the case of low battery, and no internal readout of the battery level.

Other

Some additional future tasks include adding a way to create project folders where recordings and configuration files could be saved in a manner similar to the B&K file path system on their type 2250 sound meter. The user interface could also be improved, making it easier to use and displaying additional useful information. It may also be possible to steer the beam and listen to it at a particular aiming. This is something that is to be assessed in the future.

References

1. MEGGER, Acoustic Imager: 132.5 dBA Max Acoustic Measurement, Grainger. [Online]. Available: <https://www.grainger.com/product/MEGGER-Acoustic-Imager-132-5-dBA-833TE3>. [Accessed: May 4, 2025].
2. Fluke Corporation, Fluke ii915 Industrial Acoustic Imager, Transcat. [Online]. Available: <https://www.transcat.com/fluke-ii915-industrial-acoustic-imager>. [Accessed: May 4, 2025].
3. NTi Audio, Acoustic Camera CAM IV64, NTi Audio. [Online]. Available: <https://us.shop.nti-audio.com/products/acoustic-camera-cam-iv64>. [Accessed: May 4, 2025].
4. H. H. H. Home, Microphone Arrays: Theory and Applications, Univ. of Toronto. [Online]. Available: <https://www.waves.utoronto.ca/prof/svhum/ece422/notes/15-arrays2.pdf>. [Accessed: May 4, 2025].
5. Marshall Bruner, Phased Arrays - Steering and the Antenna Pattern | An Animated Intro to Phased Arrays, YouTube, Feb. 1, 2025. [Online video]. Available: <https://www.youtube.com/watch?v=jSDLfcNhThw>. [Accessed: May 4, 2025].
6. Canonical Ltd., "Package Management," Ubuntu Server Guide, Ubuntu, 2024. [Online]. Available: <https://ubuntu.com/server/docs/package-management>
7. The Git Community, "Git Documentation," Git SCM, 2024. [Online]. Available: <https://git-scm.com/doc>
8. OpenCV.org, 2024, "OpenCV: Open Source Computer Vision Library," OpenCV, [Online]. Available: <https://opencv.org>

Appendix

Bill of Materials

Table 11: Bill of Materials for Each Array Board

Reference	Value	P/N	Footprint	Qty
C1, C3	1uF	GRM033R60J105MEA2D	Resistor_SMD:R_0603_1608Metric	2
C2, C4, C5, C6	100nF	CL05B104KP5VPNC	Capacitor_SMD:C_0402_1005Metric	4
J1	Conn_01x05_Pin	B5B-XH-A	Connector_JST:JST_XH_B5B-XH-A_1x05_P2.50mm_Vertical	1
MK1, MK2, MK3, MK4	n/a	IM69D120V01XTSA1	IM69D120V01XTSA1:XDCR_IM69D120V01XTSA1	4
R1, R2, R3, R4	33 ohms	CRCW060333R0FKEA	Resistor_SMD:R_0603_1608Metric	4
R5, R6, R7, R8	100 ohms	ERJ-3EKF1000V	Resistor_SMD:R_0603_1608Metric	4
U1	n/a	74LCX125MTCX	74LCX125MTCX:SOP65P640X120-14N	1

License

MIT License

Copyright (c) 2025 Jesse Kaye

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

NOTE: THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.